# A scheme for predicting user-perceived networked application performance

**M A Oliver, I W Phillips, D J Parish and K R Bharadia**

*Attempting to launch applications over an under-provisioned best-effort network often leads to wasteful, unsuccessful traffic being forced on to the network and user time being lost as a consequence of aborted connections. Such traffic will only contribute to the network load. A network monitoring agent which not only measures the network loading, but also provides an indication to an end user about how a specific application is likely to perform, enables application-start decisions to be made before the application is actually run.*

## 1. Introduction

When running an Internet application between local and remote sites, the user has no advance knowledge of how well the application is likely to behave. If, due to excessive network loading, the application performance is poor, the user may terminate the session. This leads to unnecessary traffic being forced on to the network, wasting network resources and user time. The effects of congestion on a network can manifest itself in three physically quantifiable forms — packet loss, packet delay and packet delay variation [1]. Each of these affects the quality of network applications; the magnitude of this effect is application specific.

By applying network monitoring techniques which measure loss and delay across a network, it is possible to know the status of the network loading at any one time [2]. However, this information does not of itself allow network operators (or users) to know how specific applications would behave under such load conditions. This is because the relationship between network and application performance is often complex. It can, however, be determined experimentally.

An agent has been developed at Loughborough University in conjunction with BT and EPSRC, which measures network loading statistics between itself and a remote site. Using these statistics and profiles of how application performance diminishes with increased network loading, the performance of an application can readily be estimated. This predicted performance of applications can be made available to users who can then decide whether or not to run the application. If the predicted performance is likely to be poor, the user could either accept poor performance, or wait until the loading subsides in order to improve the perceived quality of service. The latter will lead not only to the conservation of network resources, but also to the potential saving of user time.

This paper describes the operation of the agent, and the processes by which it is able to predict the quality of an application session. Conceptually this is achieved by the agent transmitting (and subsequently receiving) test traffic to a remote destination. This traffic could use a protocol similar to that of the application, such as user datagram protocol (UDP) or transmission control protocol (TCP), or more generally Internet control management protocol (ICMP). Observing the behaviour of this traffic, such as round-trip time, loss, or possibly single-way delay, enables approximations for the loss and delay statistics of the network to be made. Prior to the agent running, a number of applications will have individually had their performance empirically graded under several different conditions (points) of controlled network loading. The predicted application performance for the measured loading statistics is then determined, by essentially interpolating between these known points. The user is then informed of the predicted application performance.

## 2. Grading application sensitivity

Different applications have different network requirements and hence exhibit different performance characteristics when subjected to a specified condition of network load. A set of empirical sensitivity profiles therefore needs to be built up for each application that is likely to run across the network. This section briefly outlines the techniques required for empirically grading the behaviour of applications against network load.

One of the problems associated with performing such grading tests is that it is very difficult to set up a wide range of controlled network loading characteristics on a live network. Hence a loaded network emulator (LNE) [3], which replicates loading conditions by re-creating

controlled conditions of loss, delay and delay variation, was designed for this process.

To perform such tests, two workstations are connected to a switch — one directly, the other in series with the LNE emulating a network link between the two workstations. This configuration can then be programmed to be representative of any given network with the loading varied from low to high at will. An application is run between the two workstations, via the LNE replicating varying controlled degrees of loading, and the application performance is assessed. Some applications can be objectively assessed, for example by assigning empirical grades according to download times in the case of a Web-browsing session. Others require subjective grading, for example the assessment of the video quality produced during a videoconferencing session.

For subjectively graded applications, a series of user trials is carried out. At the start of each trial the subject is shown the application running when there are no loading effects emulated by the LNE. This enables the subject to know how the application would behave under ideal conditions. The subject is then required to assess a series of impairment tests. In each test the subject is shown two test samples; one where the application is run through an unloaded network and the other where the application is run against various degrees of load (the user is not informed which run corresponds to the unimpaired case). The user awards the session perceived to be impaired a grade betwen 1 and 5 loosely based around the principles of the CCIR 5-Point Impairment Scale [4], where the grades '1' to '5' respectively correspond to 'bad', 'poor', 'average', 'good' and 'excellent' performance. The session perceived as unimpaired is given a mark of 5. This process is repeated for a range of values of loss ratio, delay and delay variation; the latter two quantities are obtained by specifying the required mean and standard deviation of delay when configuring the LNE. Tests are carried out with several users so that a mean opinion score (MOS) for each loading point can be calculated.

Unfortunately, the measured values of loss ratio, average delay and variation of delay on a live network are highly unlikely to exactly match one of the network loading points used in the assessment. Subsequently, techniques have been developed to provide an algebraic representation of the grading profile, the simplest profile taking the general form:

$$G(r, d, v) = a_0 + a_1 r + a_2 d + a_3 v + a_4 rd + a_5 rv + a_6 dv + a_7 rdv \quad \text{..... (1)}$$

where $r$ is a function of loss ratio, $d$ is a function of the mean delay and $v$ is a function of the standard deviation of delay. The coefficients $a_0$ to $a_7$, which are computed using the technique shown in the Appendix, describe how the empirical grade of an application varies with the three parameters.

Such profiles provide a simple empirical approximation for assessing how application performance is likely to vary with changes in network loading, and are satisfactory for certain applications. However, other applications have a more complicated grading profile which can be accommodated by a higher-order expression.

In the case of a typical videoconference package, where pure delay does not directly degrade the quality of the received audio and video (although excessive latency will affect usability), the predicted grade can be computed as a function of $r$ and $v$:

$$G(r, v) = b_0 + b_1 r + b_2 r^2 + b_3 v + b_4 rv + b_5 r^2 v + b_6 v^2 + b_7 rv^2 + b_g r^2 v^2 \quad \text{..... (2)}$$

The technique outlined in the Appendix can be extended to generate the $b_i$ coefficients.

The $a_i$ coefficient for a typical Web-browsing session and that for an Internet-game are tabulated in Table 1, along with the $b_i$ coefficients for a typical videoconferencing session. Using equation (1) or (2) as applicable, the tabulated coefficient values and the measured network loading statistics, it is possible to predict how well a specific application is likely to perform. It should be noted that, in a practical implementation of the system, a specific manufacturer's application products would be graded rather than simply use a generic representation.

## 3. Details of the network monitoring agent

### *3.1 Agent topology*

An agent has been constructed which is based around a TCP server [5] running under the Linux operating system. Other implementations of the agent are possible, using different protocols, but all will be similar in principle. Its primary role is to monitor network loading and give feedback on how applications are likely to perform under the current network loading.

TCP clients reside on workstations in the vicinity of the agent. The primary function of the client is to request information about the network loading between itself and the agent (which should usually be low for a well-managed client network) and the loading between the agent and the remote station (or site if appropriate). It also has the facility to request information detailing how specific applications are likely to behave.

Table 1 Coefficients for equations (1) and (2) for different applications.

| Web browsing | | Typical Internet virtual-reality game | | Typical Internet videoconference | |
|---|---|---|---|---|---|
| Parameter<br>$r$ = packet loss ratio<br>$d$ = mean delay (ms)<br>$v$ = std delay (ms) | | Parameter<br>$r$ = packet loss ratio<br>$d$ = mean delay (ms)<br>$v$ = std delay (ms) | | Parameter<br>$r = \log_{10}$ (packet loss ratio)<br>$v$ = std delay (ms) | |
| Coefficient | Value | Coefficient | Value | Coefficient | Value |
| $a_0$ | 5 | $a_0$ | 5.0 | $b_0$ | −1.8 |
| $a_1$ | −9 | $a_1$ | −3.5 | $b_1$ | −4.3 |
| $a_2$ | −0.007 | $a_2$ | $-4.8 \times 10^{-3}$ | $b_2$ | −0.53 |
| $a_3$ | 0 | $a_3$ | $-3.7 \times 10^{-3}$ | $b_3$ | $-3.0 \times 10^{-3}$ |
| $a_4$ | 0.09 | $a_4$ | $6.9 \times 10^{-4}$ | $b_4$ | $5.2 \times 10^{-4}$ |
| $a_5$ | 0 | $a_5$ | $-5.0 \times 10^{-4}$ | $b_5$ | $6.0 \times 10^{-5}$ |
| $a_6$ | 0 | $a_6$ | $-4.5 \times 10^{-6}$ | $b_6$ | $9.6 \times 10^{-6}$ |
| $a_7$ | 0 | $a_7$ | $-1.5 \times 10^{-5}$ | $b_7$ | $8.1 \times 10^{-6}$ |
| | | | | $b_8$ | $1.0 \times 10^{-6}$ |

Figure 1 shows how the agent should be connected to a network. It can be seen that the agent is placed on the same switch as the local workstations or (more usefully) on the backbone of a site network assuming that this provides a high quality of connection between the client and agent relative to the outside world. This ensures that the network loading between the agent and a remote site is representative of the network loading between the local and remote stations.

### *3.2 Client-agent transactions*

The agent is an active TCP server which awaits requests from a client, processes the request and waits for the next request; during any idle time, the agent tests known addresses on a periodic basis to update the network loading statistics. To facilitate this operation, the agent contains two databases — the first, or primary database, is used to hold details of network applications and their loading senstivity profiles, while the secondary database holds network loading statistics between the agent and remote stations. The latter is used to rapidly obtain loading statistics between the agent and known sites if required. This is particularly important for corporate intranet use where a limited range of remote sites is frequently contacted.

To obtain data, the client can send one of two possible message types to the agent:

- a request to check the network loading, or
- a request to list the application names stored in the application database within the agent.

The latter is always automatically transmitted as part of the client initialisation process.

In response, the server respectively transmits:

- a message detailing the network loading statistics and predicted application grades, or
- one or more messages listing the applications stored in the application database.

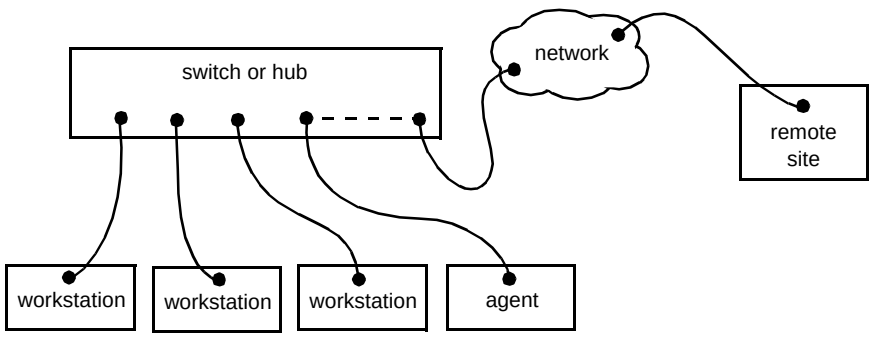


Fig 1 Representative positioning of an agent.

### *3.3 Message formats*

As suggested above, four different message formats are used in the client-agent transactions; these formats are detailed in Fig 2, and have two common features. Firstly, they have an authentication field so that any bogus messages can be eliminated. The second is a request-type identifier so that the nature of the message can be investigated. A summary of possible request-type identifiers is listed in Table 2.

Table 2 Function of the request type field.

| Request type | Function |
|---|---|
| Client loading request | Prompt agent to obtain loading statistics and/or grades |
| Client applications request | Obtain list of application names known to agent |
| Server loading reply | Transmit loading statistics and grades to client |
| Server applications reply | Transmit application names to client |

The 'client loading request' message, used for requesting network loading information from the agent, has fields which indicate how many IP addresses need to be tested along with two or more IP addresses. The first IP address is that of the local workstation, where the client resides. The second IP address is that of a remote site. Provision for extra IP addresses exists in the message for dealing with sites possessing multiple IP addresses. In addition there is an extra field which provides a reference to an application, for when the user needs to obtain the predicted performance of a specific application.

The 'server loading reply' message is transmitted from the agent in response to a 'client loading request' message. It contains the number of IP addresses that have been tested as a consequence of the request, and the predicted application performance and loading statistics for each IP address under scrutiny.

The 'client applications request' message is transmitted to the agent in order to update the list of applications which the client may ask the agent about. These are held in the application database within the agent. The number of applications currently known to the client is sent in this message so the agent knows how many application names need to be transmitted.

'Server applications reply' messages are transmitted from the agent to the client in response to the 'client applications request' message. These contain the number of additional application names held in the message, the number of names that still require transmission and the application names themselves.

### *3.4 Agent mechanism*

Figures 3 and 4 illustrate the mechanism of the client-agent-remote site transactions. Figure 3 refers to the mechanism before the agent has processed any loading

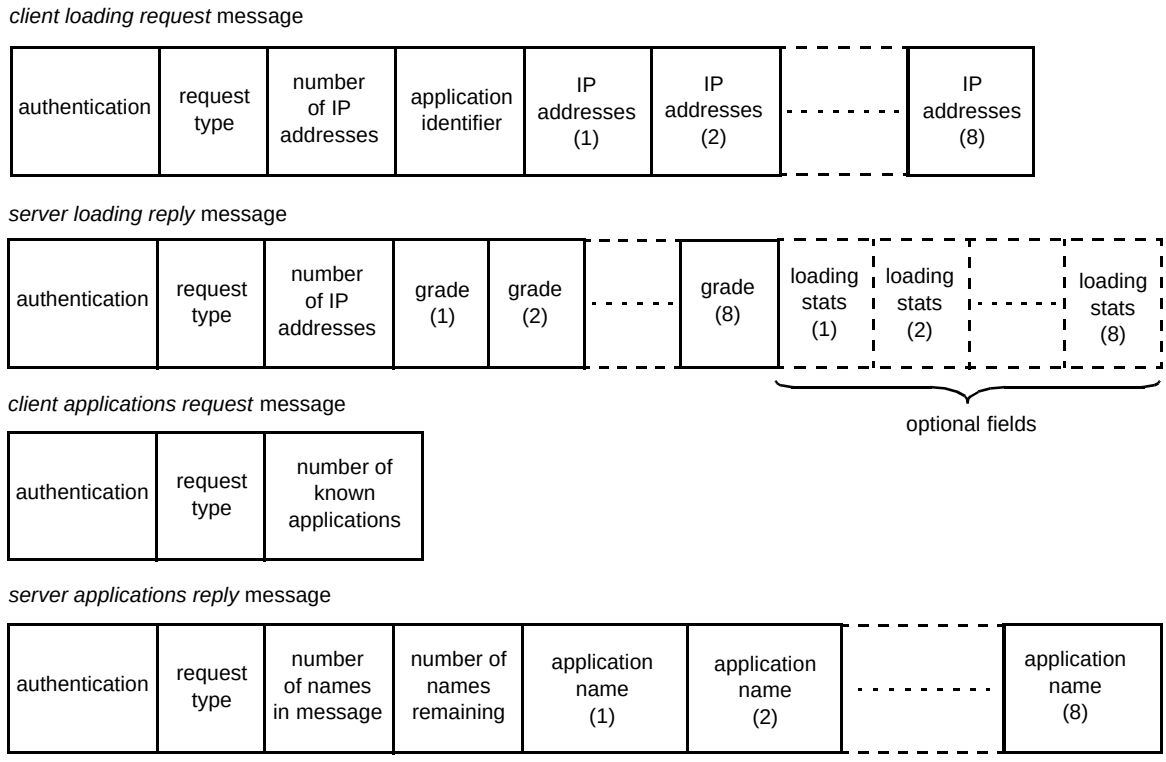


Fig 2 Message formats for client-agent requests.

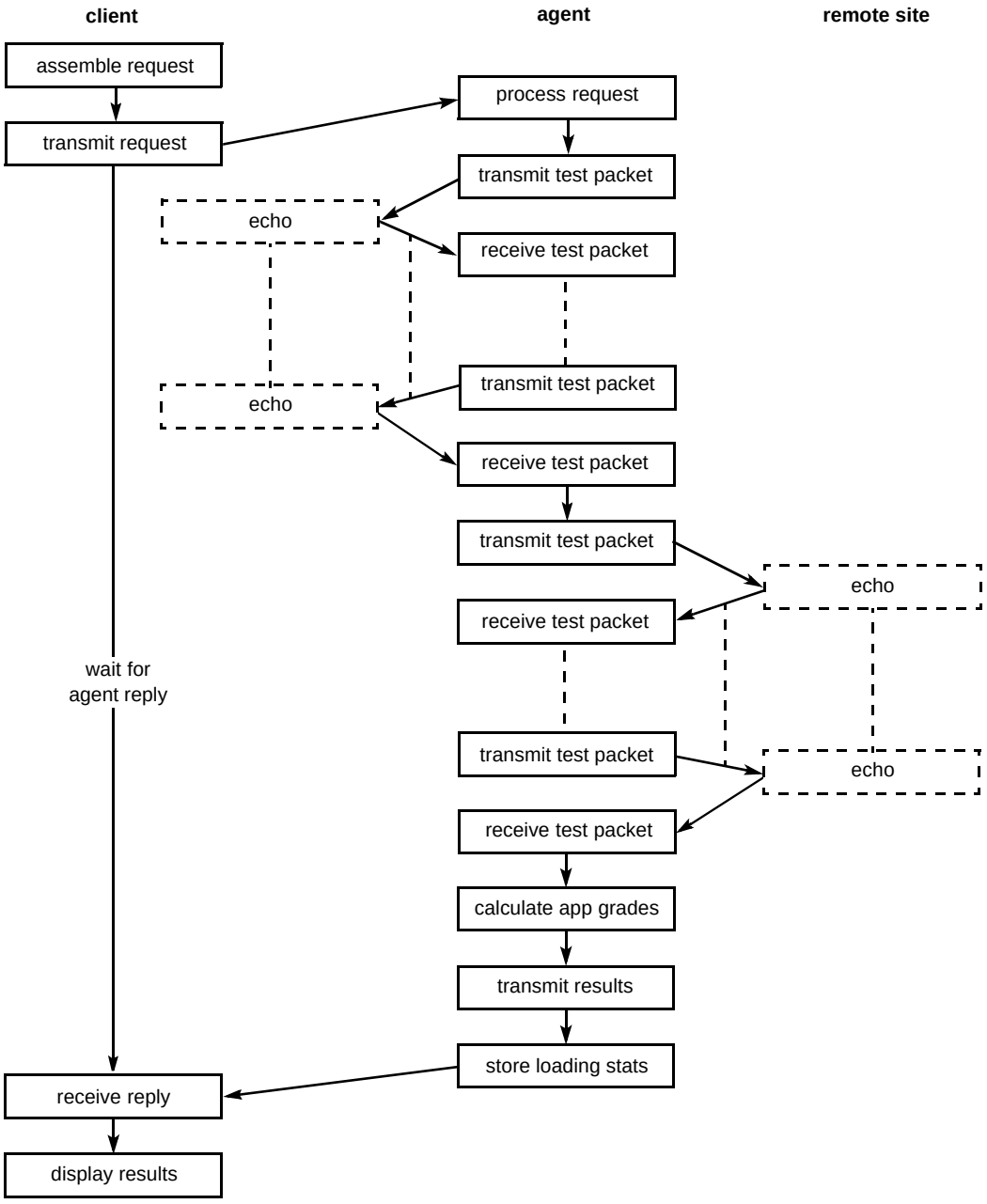


Fig 3 Agent mechanism — first request transaction.

requests, i.e. when the agent has no IP addresses stored in its database.

In order to obtain information about the network loading conditions, the user is required to enter the machine name (or IP address) of the local station, and that of a remote site. If machine names are specified, the client first resolves the respective IP address (or IP addresses in the case of certain Web sites which possess multiple IP addresses). The user has the choice of specifying whether network loading statistics or application performance grades are required. In the latter case the user specifies the name of the required application. This data is assembled into the 'client loading request' message which is transmitted to the agent as illustrated in Fig 3. The client then waits for a response from the agent.

When the 'client request' message arrives at the agent, the IP addresses are extracted from the message. Before the first loading request arrives there will be no loading statistics stored in the secondary database, so the agent transmits a set of 200 echo-request packets [5] in turn to each of the required IP addresses. For simplicity ICMP packets are used, but the scheme could be extended to other types of packet. Each of these packets is separated by a time delay of 10 ms to ensure that results can be rapidly obtained without severely affecting the server. From these tests, the network loading statistics for each required IP address are

determined; this process is discussed later. These IP addresses along with the respective loading statistics are then stored in the secondary database within the agent. If required, the agent calculates the predicted application grades from the sensitivity profiles held in the primary database and the application loading statistics, using equation (1) or (2). These results are assembled into the 'server loading reply' message for transmission to the client and dissemination to the user. If the user requires application grading, the results are given as either a mark out of five, or as a textual message based loosely on the CCIR 5-point impairment scale.

Figure 4 refers to the mechanism of the system once the agent has stored at least one IP address in its network loading (secondary) database. During any idle time, the agent measures and updates the network loading statistics for all IP addresses held in this database in a 'round-robin'

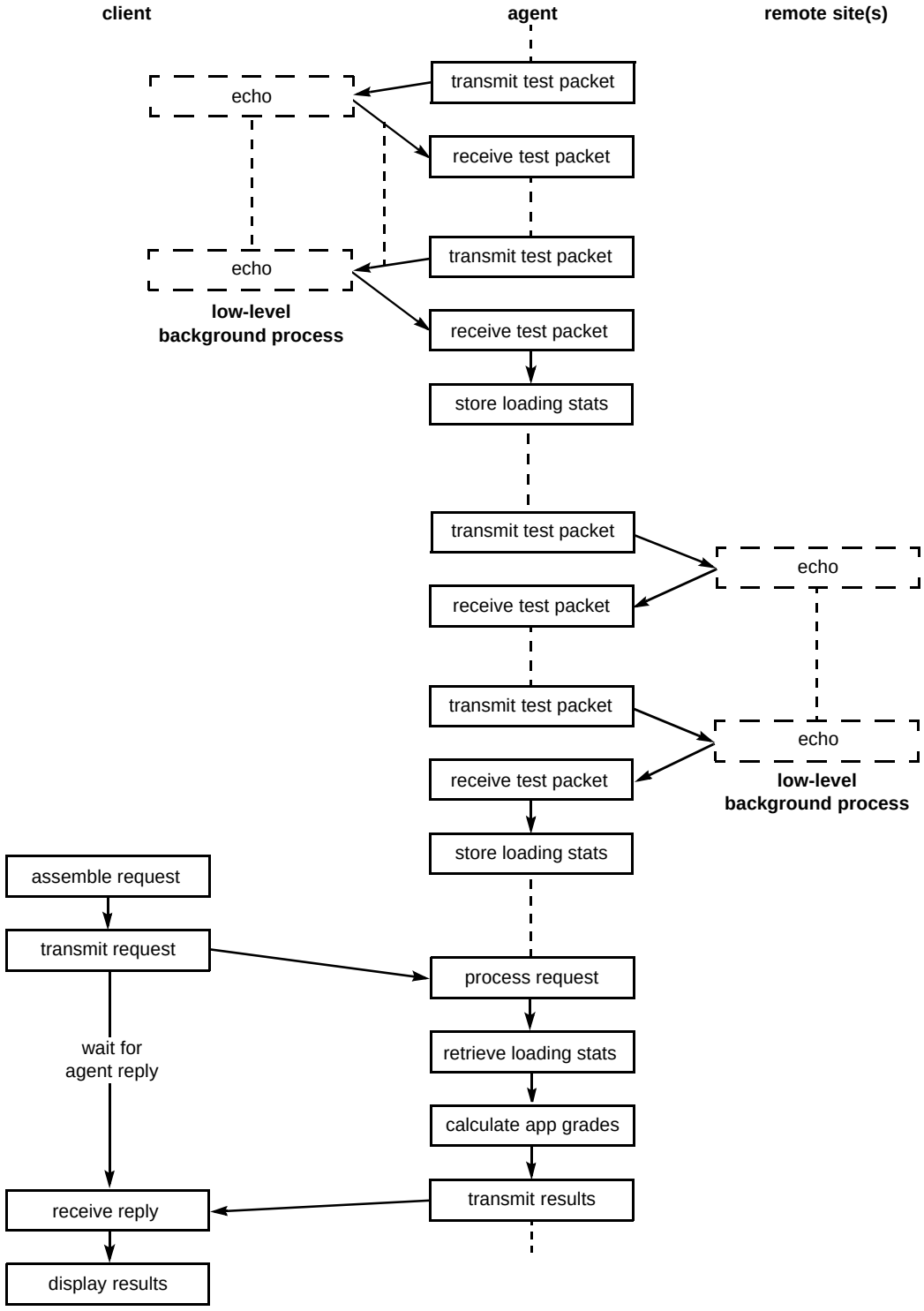


Fig 4 Agent mechanism — further transactions.

manner, placing a gap of several seconds between site tests. This is to ensure that the statistics held in the database are representative of the current network loading conditions. When the agent receives subsequent 'client loading request' messages it checks its database for existence of the required IP addresses. If any of the IP addresses exist, the loading statistics are extracted directly from the database, hence saving time — otherwise the IP addresses in question are tested in the usual manner.

The secondary database, which holds IP addresses and loading statistics, may be enhanced to aid the scalability of the system, and prevent the agent itself forcing unnecessary traffic on to a network. This can be achieved by splitting the database into three sections. The first section will contain a list of important IP addresses preprogrammed by a network manager, which could be especially useful in corporate intranet situations. The second will contain a list of the most frequently accessed sites, not lying in the preprogrammed section, for which network loading data is requested. The final section will contain the IP addresses and loading statistics for the most recently accessed sites. The second and third sections of the database will therefore dynamically adapt to current usage.

### *3.5 Evaluating network performance*

In order to measure the network loading between the agent and a remote site, the agent transmits a total of $n_{out}$ ICMP echo-request packets to the remote site. From the round-trip times and observations of packet loss, the network loading characteristics can be determined. To perform the network measurement calculations, an assumption is made that the network is ideally symmetric, i.e. packets travelling from the agent to the remote site will be perturbed in the same way as packets travelling from the remote site back to the agent.

For each ICMP packet received, its delay through the network can be approximated as half of its round-trip time. With knowledge of the delay times for all received packets, the mean delay, standard deviation of delay, along with the maximum and minimum delays can be readily found.

A little more care needs to be exercised when estimating the packet loss ratio, *R*, across a network. *R* cannot simply be calculated as half of the packet loss ratio, but requires further analysis as Fig 5 illustrates.

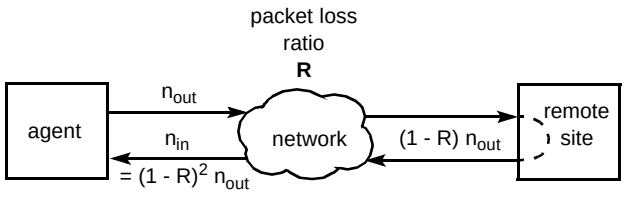


Fig 5 Determining the packet loss ratio.

Suppose the agent transmits $n_{out}$ packets to the remote site. For a network packet loss ratio, *R*, a total of $n_{remote}$ packets are likely to reach the remote site where:

$$n_{remote} = (1 - R) n_{out} \quad \text{..... (3)}$$

$n_{remote}$ packets will then be echoed from the remote station back to the agent via the network. Suppose $n_{in}$ packets return to the agent, then:

$$n_{in} = (1 - R) n_{remote} = (1 - R)^2 n_{out} \quad \text{..... (4)}$$

Rearranging equation (4) and solving for the packet loss ratio, *R*, gives:

$$R = 1 - \sqrt{\frac{n_{in}}{n_{out}}} \quad \text{..... (5)}$$

## 4. Example

To examine the effectiveness of the network monitoring agent, a videoconference session was established between two workstations under network conditions of light and heavy loading. The results of these are respectively illustrated in Figs 6 and 7. Both figures are screen dumps, showing how the videoconference session is affected by network load, and demonstrate how well the agent can identify good and bad loading conditions. It should be noted that the video sequence received from the remote station, rather than the local video, is displayed in the video window.

In both tests the client interrogated the agent to determine the network loading conditions, and hence ascertain how well the application is likely to perform. In the case of light loading, the agent predicted that the loading between the agent and the local station was minimal, and as such the application agent would perform well if run between the agent and the local station. The agent also reported that the application would again perform well if the video-conference was run between the agent and the remote site. These results are substantiated in Fig 6 by the excellent quality of the received video snapshot.

In the case of heavy network loading, the agent again predicted light loading between the local station and the agent. This is to be expected as the agent lies in the vicinity of the local station; bad performance here would indicate a problem with the local side of the network. However, the agent predicted that the performance of the application if run between the agent and the remote station would be bad. The net result is that the performance of the application was bad, as can be seen from the video snapshot of Fig 7. Given the information that the application is likely to have bad performance **before** the videoconference session is established, a user might wish not to attempt to make the connection, therefore not putting any additional stress on to the over-used network resources.

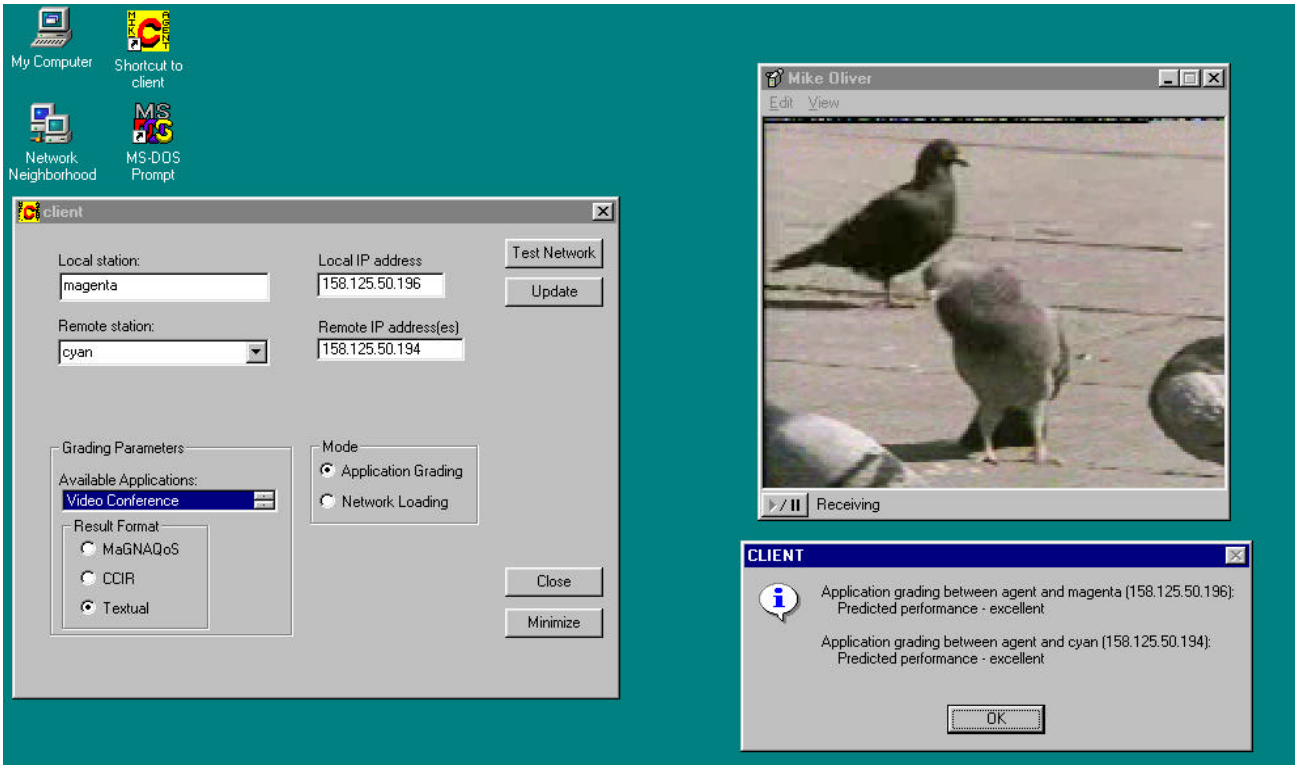


Fig 6 Agent predicting best possible videoconference performance over a lightly loaded network.

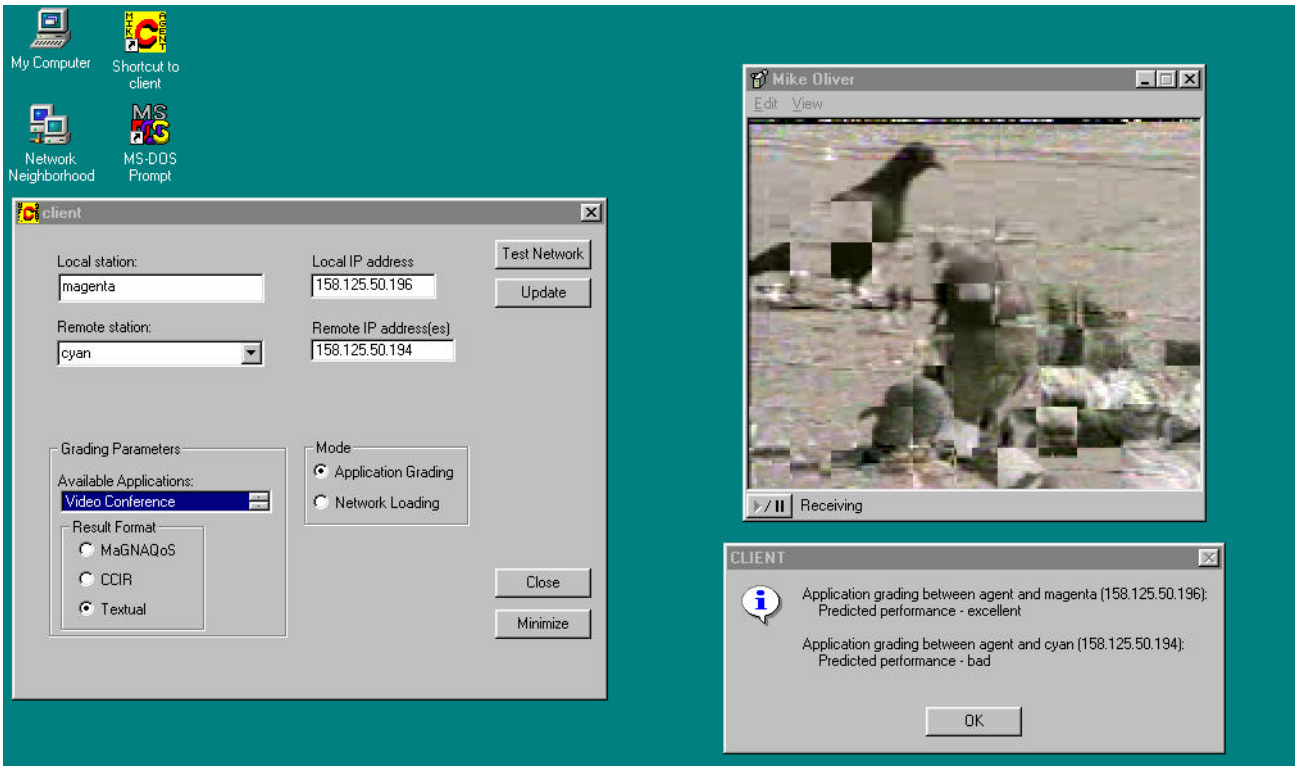


Fig 7 Agent predicting bad videoconference performance over a heavily loaded network.

## 5. Conclusions

This paper has illustrated the mechanism of a network monitoring agent and demonstrated how it may be used to predict the performance of a network application before the application is run. This has two benefits:

- firstly, a user will know in advance what to expect from the application in question if an attempt is made to run it,
- secondly, the user will also be able to make the decision whether or not to establish a connection if the

predicted performance is known in advance not to be satisfactory.

If a user chooses not to run an application due to predicted poor performance, several savings are made. The network will not be subjected to the additional load brought about by the attempt and subsequent aborting of trying to establish a bad connection. In addition, time will also be saved as a consequence of the user not waiting to establish the connection and then deciding that the performance is poor.

The potential savings, as a result of deploying a network monitoring agent in a large organisation, could become immense.

**Acknowledgements**

The authors would like to thank EPSRC and BT for providing funding towards this research, and BT for the use of the Futures Testbed. Specifically, the authors would like to thank Rob Davison, Steven Beaumont, Kevin Smith, Chris Gibbings and Tony Dann from BT Laboratories for discussions and assistance with the test bed.

**Appendix**

*Application grading profiles*

This appendix details a method for producing algebraic expressions to describe how the grade of an application varies as a function of network loading parameters, namely loss ratio ($r$), average delay ($d$) and the standard deviation of delay ($v$). The following analysis shows how to derive a first-order expression for the grade variation. This technique can readily be extended to a higher order but little benefit will be gained from showing such analysis.

In the simplest first-order case, using a set of grading points based on loss ratio, mean delay and the standard deviation of the delay, a minimal application grading profile can be obtained by using two values of each of the three network loading parameters, thus leading to 8 points which have been empirically graded as shown in Fig A1.

For the three network loading parameters, $r$, $d$ and $v$, the application grade $G(r, d, v)$ can be expressed in the general form:

$$G(r, d, v) = a_0 + a_1 r + a_2 d + a_3 v + a_4 rd + a_5 rv + a_6 dv + a_7 rdv \quad \text{..... (A1)}$$

where the $a_i$ coefficients are determined from the following analysis.

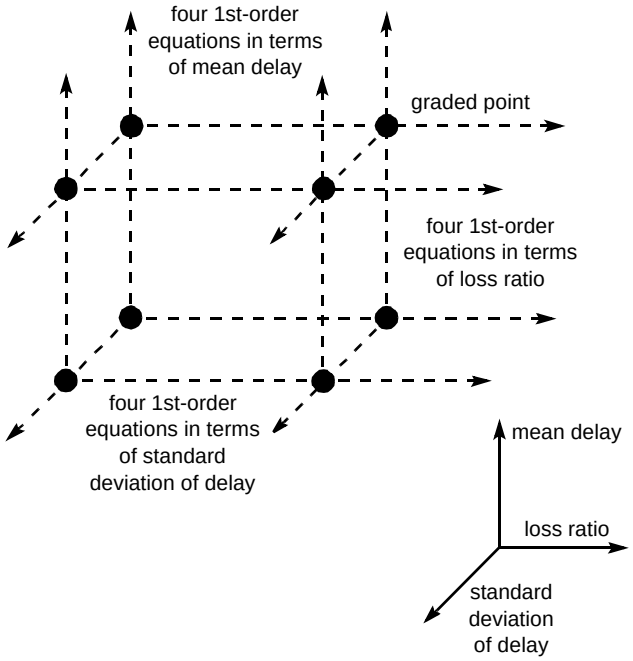


Fig A1 Creating a grading profile.

Partially differentiating equation (A1) with respect to $r$ gives:

$$\frac{\partial G(r, d, v)}{\partial r} = a_1 + a_4 d + a_5 v + a_7 dv \quad \text{..... (A2)}$$

Partial differentiation of equation (A1) with respect to the delay, $d$, results in:

$$\frac{\partial G(r, d, v)}{\partial d} = a_2 + a_4 r + a_6 v + a_7 rv \quad \text{..... (A3)}$$

Finally, partially differentiating equation (A1) with respect to $v$ gives:

$$\frac{\partial G(r, d, v)}{\partial v} = a_3 + a_5 r + a_6 d + a_7 rd \quad \text{..... (A4)}$$

With reference to Fig A1, it can be seen for known values of mean delay and the standard deviation of delay, four equations exist which describe how the grade varies with the loss ratio, $r$. These take the general form:

$$G(r, d = d_i, v = v_j) = f_{k1} r + f_{k0} \quad \begin{matrix} i = 1, 2 \\ j = 1, 2 \\ k = 2(j-1) + i \end{matrix} \quad \text{..... (A5)}$$

where $i$ is the index to the known delay value, $j$ is the index to the known standard deviation of delay value, and $k$ is the equation identifier. The $f$ coefficients can be obtained using elementary curve-fitting techniques.

Also, from Fig A1, four first-order equations exist which describe how the grade varies with $d$, for fixed values of $r$ and $v$. These equations take the general form:

$$G(r = r_i, d, v = v_j) = g_{k1}d + g_{k0} \quad \begin{matrix} i = 1, 2 \\ j = 1, 2 \\ k = 2(j-1) + i \end{matrix} \qquad \text{..... (A6)}$$

Finally, four first-order equations exist which show how the perceived grade changes with variations in $v$. These equations, for fixed values of $r$ and $d$, take the following general form:

$$G(r = r_i, d = d_j, v) = h_{k1}v + h_{k0} \quad \begin{matrix} i = 1, 2 \\ j = 1, 2 \\ k = 2(j-1) + i \end{matrix} \qquad \text{..... (A7)}$$

Partially differentiating equation (A5) with respect to $r$ gives:

$$\frac{\partial G(r, d = d_i, v = v_j)}{\partial r} = f_{k1} \quad \begin{matrix} i = 1, 2 \\ j = 1, 2 \\ k = 2(j-1) + i \end{matrix} \qquad \text{..... (A8)}$$

The partial differentiation of equation (A6) with respect to $d$ results in:

$$\frac{\partial G(r = r_i, d, v = v_j)}{\partial d} = g_{k1} \quad \begin{matrix} i = 1, 2 \\ j = 1, 2 \\ k = 2(j-1) + i \end{matrix} \qquad \text{..... (A9)}$$

Finally partial differentiating equation (A7) with respect to $v$ gives:

$$\frac{\partial G(r = r_i, d = d_j, v)}{\partial v} = h_{k1} \quad \begin{matrix} i = 1, 2 \\ j = 1, 2 \\ k = 2(j-1) + i \end{matrix} \qquad \text{..... (A10)}$$

Equating expressions (A2) and (A8) produces four equations which take the following form:

$$a_1 + a_4d_i + a_5v_j + a_7d_iv_j = f_{k1} \quad \begin{matrix} i = 1, 2 \\ j = 1, 2 \\ k = 2(j-1) + i \end{matrix} \qquad \text{..... (A11)}$$

Equating expression (A3) with expression (A9) leads to:

$$a_2 + a_4r_i + a_6v_j + a_7r_iv_j = g_{k1} \quad \begin{matrix} i = 1, 2 \\ j = 1, 2 \\ k = 2(j-1) + i \end{matrix} \qquad \text{..... (A12)}$$

Finally equating expression (A4) with expression (A10) yields four equations having the following form:

$$a_3 + a_5r_i + a_6d_j + a_7r_id_j = h_{k1} \quad \begin{matrix} i = 1, 2 \\ j = 1, 2 \\ k = 2(j-1) + i \end{matrix} \qquad \text{..... (A13)}$$

Also, it is known that:

$$G(r_1, d_1, v_1) = a_0 + a_1r_1 + a_2d_1 + a_3v_1 + a_4r_1d_1 + a_5r_1v_1 + a_6d_1v_1 + a_7r_1d_1v_1 \qquad \text{..... (A14)}$$

Combining equations (A11) to (A14) and ignoring redundant expressions leads to a matrix equation of the form:

$$\begin{bmatrix} 1 & r_1 & d_1 & v_1 & r_1d_1 & r_1v_1 & d_1v_1 & r_1d_1v_1 \\ 0 & 0 & 0 & 1 & 0 & r_1 & d_1 & r_1d_1 \\ 0 & 0 & 0 & 1 & 0 & r_2 & d_1 & r_2d_1 \\ 0 & 0 & 0 & 1 & 0 & r_1 & d_2 & r_1d_2 \\ 0 & 0 & 0 & 1 & 0 & r_2 & d_2 & r_2d_2 \\ 0 & 0 & 1 & 0 & r_1 & 0 & v_1 & r_1v_1 \\ 0 & 0 & 1 & 0 & r_2 & 0 & v_1 & r_2v_1 \\ 0 & 1 & 0 & 0 & d_1 & v_1 & 0 & d_1v_1 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \\ a_4 \\ a_5 \\ a_6 \\ a_7 \end{bmatrix} = \begin{bmatrix} G(r_1, d_1, v_1) \\ h_{11} \\ h_{21} \\ h_{31} \\ h_{41} \\ g_{11} \\ g_{21} \\ f_{11} \end{bmatrix} \qquad \text{..... (A15)}$$

or:

$$\Phi A = \beta \qquad \text{..... (A16)}$$

So, the unknown $a_i$ coefficients can be found from:

$$A = \Phi^{-1}\beta \qquad \text{..... (A17)}$$

## References


1 Lee B G, Kang M and Lee J: 'Broadband telecommunications technology', Artech House Inc (1993).

2 Bashir O, Phillips I, Parish D, Adams J L and Spencer T: 'The management and processing of network performance information', BT Technol J, 16, No 4, pp 203—212 (October 1998).

3 Oliver M A, Phillips I W, Parish D J and Bharadia K: 'Determining application sensitivity to network loading effects', Irish Signals and Systems Conference, ISSC '98, Dublin Institute of Technology, pp 117—124 (1998).

4 Daumer W R: 'Subjective evaluation of several efficient speech coders', IEEE Transactions on Communications, COM—30, No 4, pp 655—622 (1982).

5 Stevens W R: 'Unix network programming', Prentice Hall Inc, Englewood Cliffs, New Jersey (1990).

Mike A Oliver gained his BEng degree in Electronic and Electrical Engineering at Loughborough University of Technology in 1992. In 1997 he received the PhD in Digital Controller Error Analysis from Loughborough University.

From 1997 he has worked as a research associate, funded by EPSRC and BT, in the High Speed Networks Research group at Loughborough University.

His current work relates to predicting application performance over loaded networks.

Iain Phillips received his BSc and PhD degrees in Computer Science from the University of Manchester with his PhD research related to Load Balancing on Massively Parallel Computers. From 1992 he has worked as a Research Assistant in the High Speed Networks Group on work related to broadband network performance monitoring, teleconferencing over ATM, video error concealment, ATM-based surveillance systems, and systems integration. This work has been funded by EPSRC, RACE (Europe) and BT. Specifically, his current research is on the performance monitoring and measurement of high-performance networks under the BT University Research Initiative (URI) using SuperJanet. He is a Graduate Student Member of the British Computer Society.

David J Parish holds BSc and PhD degrees from the University of Liverpool He has worked as a Scientific Officer at the UKAEA Culham Laboratory and as a Demonstrator in the Electrical Engineering Department at Liverpool University.

From 1983 he has held the position of Lecturer, Senior Lecturer and now Reader in the Department of Electronic and Electrical Engineering at Loughborough University.

His research interests concern the management, operation, monitoring and application of high performance networks. Specifically, he leads Loughborough's input to the BT-funded research programme into the management of high-speed networks using SuperJanet.

Ketan R Bharadia received his BSc in 1988 from Loughborough University of Technology.

He has worked in industry for nine years for various organisations including a mobile telephone company.

Since 1997 he has worked as a PhD research student in the High Speed Networks Research group at Loughborough University, working on techniques to detect applications running over networks by statistically analysing traffic flows.